

60. Faire glisser le décor

L'écran ne montre qu'un morceau de la carte ; `defiler()` choisit lequel, au pixel près.

1. Ce qu'il faut comprendre

L'écran fait **20 cases** de large. Mais la carte du fond, dans la console, en fait **32** : 256 pixels. On ne voit jamais qu'un **morceau** de cette carte, comme par une fenêtre.

Ce qui est nouveau ici : `defiler(x, y)`. Elle dit à la console **où poser la fenêtre** sur la carte, en pixels.

`defiler(0, 0)` : on voit les colonnes 0 à 19. `defiler(8, 0)` : la fenêtre a glissé d'une case, on voit les colonnes 1 à 20. `defiler(3, 0)` : trois pixels, entre deux cases — le glissement est doux.

On ne redessine rien : le décor est posé **une fois**, écran éteint, sur les 32 colonnes (`c < 32`, et non `c < 20`). Faire défiler ne coûte qu'**une écriture** par image, comme la palette du chapitre 7. C'est pourquoi les jeux Game Boy défilent sans ralentir.

Les chiffres 0 à 7, un toutes les 4 colonnes, disent où l'on est sur la carte. Pousse DROITE longtemps : après le 7, le 0 revient. `sx` est un octet : après 255, il repart à 0, et la carte, qui fait justement 256 pixels, **boucle** au même moment. La règle de l'octet (chapitre 1) et la taille de la carte vont ensemble.

Le héros ne bouge pas : c'est un lutin, et les lutins ne défilent pas avec le fond. Rester au milieu pendant que le monde passe, c'est l'illusion de tous les jeux de course. En revanche, le texte du haut **défile avec le reste** : il est dans le fond. Pour un score qui reste en place, on se sert du **panneau** (`textePanneau`), une seconde couche que `defiler()` ne touche pas.

À toi : fais défiler aussi en hauteur avec HAUT et BAS (`sy`, et `defiler(sx, sy)`).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **915 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	60 o	← la console, à l'allumage — la mise en route de la console
main()	175 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), nombre() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	368 o	← CopierTuiles — 21 lettres sur 43, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins
"GAUCHE DROITE"	14 o	← texte() — un texte gravé

Lettres gravées : ACDEGHIORTU0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

poser()	× 2	26 o sur place, dans main()	+ AdresseCase (10 o)
nombre()	× 1	25 o sur place, dans main()	+ AdresseCase (10 o), EcrireNombre (33 o)
sprite()	× 1	23 o sur place, dans main()	
ecran()	× 2	21 o sur place, dans main()	+ AttendreVBlank (16 o)
texte()	× 1	11 o sur place, dans main()	+ EcrireTexte (10 o)
bouton()	× 2	10 o sur place, dans main()	+ LireManette (37 o)
defiler()	× 1	7 o sur place, dans main()	
image()	× 1	6 o sur place, dans main()	+ AttendreImage (28 o)

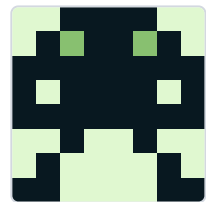
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : GAUCHE / DROITE font glisser le décor ; le héros reste au milieu.
// Des chiffres 0 à 7 sur le sol disent où l'on est.
//
// Ce qui est nouveau : defiler(x, y).
// L'écran montre 20 colonnes, mais la carte du fond en a 32 (256 pixels).
// defiler(x, y) dit où poser la « fenêtre » sur la carte, en pixels.
// defiler(0, 0) : on voit les colonnes 0 à 19. defiler(8, 0) : les colonnes 1 à 20.
// Rien n'est redessiné : une seule écriture par image.
```

```
#include <Tuile>      // un dessin de 8 × 8 pixels
#include <ecran>      // éteint ou rallume l'écran
#include <texte>      // écrit un texte à l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <nombre>     // écrit un nombre en chiffres
#include <bouton>     // lit un bouton de la manette
#include <defiler>    // fait glisser tout le fond
#include <sprite>     // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {
    "..####..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##...##",
};
```



à l'écran

3. UN DESSIN — SOL

```
Tuile SOL = {
    "33333333", "12121212", "21212121", "22222222",
    "22222222", "22222222", "22222222", "22222222",
};
```



à l'écran

4. UNE VARIABLE — SX

```
uint8_t sx = 0;      // où est la fenêtre sur la carte, en pixels : 0 à 255
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    texte(1, 1, "GAUCHE DROITE");
    for (uint8_t c = 0; c < 32; c++) {      // TOUTE la carte : 32 colonnes (pas 20)
        poser(c, 12, SOL);                 // deux rangées de sol,
        poser(c, 13, SOL);
        if (c % 4 == 0) nombre(c, 10, c / 4, 1); // 0 en colonne 0, 1 en 4, ... 7 en 28
    }
    ecran(1);                               // On rallume.

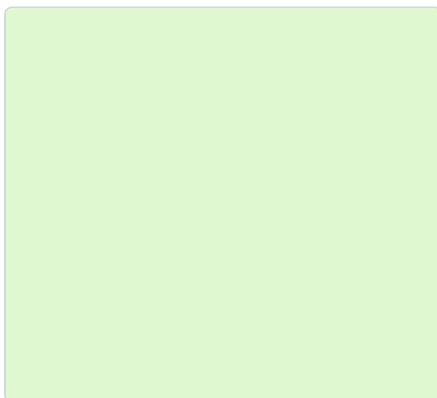
    while (true) {                          // La boucle du jeu :
        image();

        if (bouton(DROITE)) sx++;          // la fenêtre glisse vers la droite : le décor part à gauche
        if (bouton(GAUCHE)) sx--;          // 0 - 1 donne 255 : la carte boucle aussi à l'envers
                                            // (sx va de 0 à 255 et la carte fait 256 pixels :
                                            // après 255, on revient pile au début)

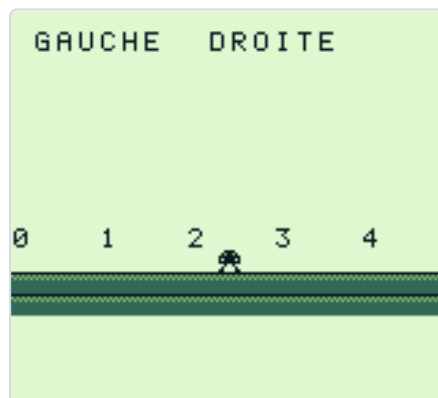
        defiler(sx, 0);                    // une seule écriture par image
        sprite(0, 76, 88, HEROS);          // le héros ne bouge pas : les lutins ne défilent pas
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



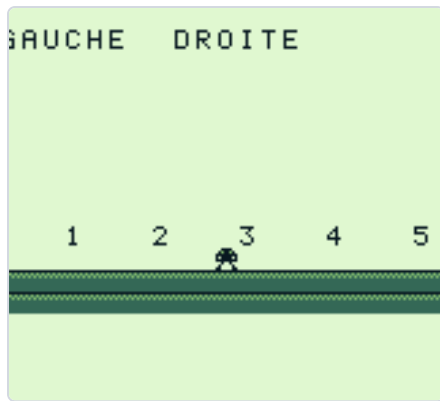
la toute première image — la console vient de démarrer — image 1



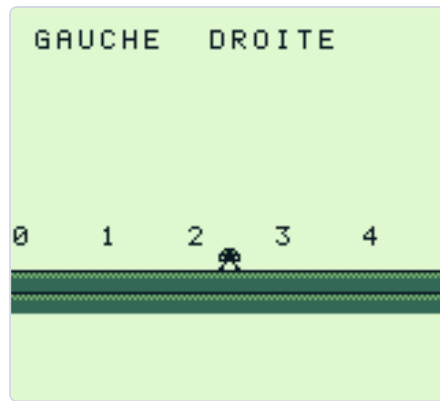
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un sol et des chiffres de 0 à 7 ; DROITE fait glisser le décor vers la gauche, et après le 7 le 0 revient.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, la fenêtre est en 0
- ✓ DROITE la fait glisser d'un pixel par image (40)
- ✓ après 255, elle repart à 0 : la carte boucle (14)
- ✓ le décor est posé jusqu'à la colonne 31
- ✓ le héros n'a pas bougé

← [59. Un petit air qui joue tout seul](#)

[sommaire](#)

[61. Une carte plus grande que l'écran : la caméra](#) →